Buscaminas en MARIE

Nicolás Naranjo
POLI
Universidad San Francisco de Quito
Quito, Ecuador
nnaranjor@estud.usfq.edu.ec

Juan David Burgos
POLI
Universidad San Francisco de Quito
Quito, Ecuador
jburgosl@estud.usfq.edu.ec

Felipe Bohórquez
POLI
Universidad San Francisco de Quito
Quito, Ecuador
fbohorquez@estud.usfq.edu.ec

I. Introducción

MARIE.js es un simulador de arquitectura educativa que permite ejecutar programas en lenguaje ensamblador sobre una arquitectura von Neumann básica. Este proyecto implementa el juego Buscaminas en un tablero de 16×16 celdas, donde la interacción con el usuario se realiza mediante ingreso secuencial de coordenadas (fila, columna, acción). El tablero inicial es generado por un script en Python que produce el mapa de minas y valores en el formato requerido por MARIE. La visualización del juego se realiza mediante colores en el display de 16×16 pixeles, representando los estados de las celdas (oculta, revelada, bandera, mina).

II. Metodología

La metodología consiste en dos partes: generación del tablero con Python y lógica del juego en MARIE. El script en Python define las coordenadas de las minas, calcula los valores de voisinas para cada celda y genera el bloque de memoria. El programa en MARIE valida las coordenadas ingresadas, actualiza el tablero visible y verifica victoria o derrota.

El flujo del juego opera en un ciclo continuo: el usuario ingresa tres valores (fila, columna, acción), el programa valida que las coordenadas estén dentro del rango válido, ejecuta la acción solicitada, actualiza el display y verifica si el juego terminó.

Validación de coordenadas: VerificaInput recibe la fila, columna y acción. Primero verifica que fila y columna estén en el rango 0-15. Si están fuera, rechaza la jugaday retorna «invalid». Si son válidas, acepta la acción (1=revelar, 2=marcar).

Algoritmo 1. VerifyInput

```
▷ r: row, c: column, a: action (1=reveal, 2=flag)
function VerifyInput(r, c, a)
    if r < 0 then
        return invalid
    end
    if c < 0 then
        return invalid
    end
    if r ≥ 16 then
        return invalid
    end
    if c ≥ 16 then
        return invalid
    end
    if a == 1 then
        return reveal
    else if a == 2 then
        return flag
    end
end
```

Conversión de índices: El display de 16×16 usa direccionamiento lineal (0-255). UpdateBoardIdx convierte (fila, columna) a índice lineal multiplicando la fila por 16 y sumando la columna.

Algoritmo 2. UpdateBoardIdx

```
▷ r: row index (0-15), c: column index (0-15)
▷ i: linear index result, k: loop counter for rows
function UpdateBoardIdx(r, c)
    i ← 0
    k ← 0
    while r > k do
        i ← i + 16
        k ← k + 1
    end
    i ← i + c
    return i
end
```

Revelado de celdas: ActionReveal convierte las coordenadas 2D a índice lineal. Obtiene el puntero del pixel mediante UpdatePixelPtr. Si la celda está oculta (COLOR_UNKNOWN = 2108), lee el valor real del tablero. Si el valor es −1 (mina), el jugador pierde. Si el valor es 0, ejecuta FloodFill para revelar automáticamente las celdas vacías adyacentes.

Algoritmo 3. ActionReveal

```
▷ r: row input, c: column input
▷ x: linear index, sr: safe revealed count
▷ value: tile value (-1=mine, 0-8), BoardPtr: board RAM address
▷ PixelPtr: display address, ColorPtr: color address
function ActionReveal(r, c, sr)
    x ← UpdateBoardIdx(r, c)
    PixelPtr ← UpdatePixelPtr(x)
    if PixelPtr == 2108 then
```

```
        BoardPtr ← D + x
        ColorPtr ← COLOR_ZERO_PTR + *BoardPtr
        *PixelPtr ← *ColorPtr
        value ← *ColorPtr − COLOR_MINE
        if value == −1 then
            return defeat
        end
        sr ← sr + 1
        if value == 0 then
            FloodFill(x)
        end
    end
    return check
end
```

Marcado de banderas: ActionFlag permite marcar o desmarcar una celda. Si la celda está oculta, cambia su color a amarillo (7FE0). Si ya tiene bandera, la remueve (vuelve a gris oscuro 2108).

Algoritmo 4. ActionFlag

```
▷ r: row input, c: column input
▷ x: linear index (BoardIdx), PixelPtr: display address
function ActionFlag(r, c)
    x ← UpdateBoardIdx(r, c)
    PixelPtr ← UpdatePixelPtr(x)
    if PixelPtr == 2108 then
        PixelPtr ← 7FE0
    end
    if PixelPtr == 7FE0 then
        PixelPtr ← 2108
    end
end
```

FloodFill: Cuando se revela una celda vacía (valor 0), FloodFill recorre el tablero buscando celdas adyacentes con valor 0 y las revela automáticamente.

Algoritmo 5. FloodFill

```
▷ x: linear index (BoardIdx)
▷ rf: repeat flood flag, BoardPtr: board RAM address
▷ PixelPtr: display RAM address, Counter: loop counter
function FloodFill(x)
    rf ← 0
    BoardPtr ← D
    PixelPtr ← F00
    while PixelPtr < 1000 do
        if PixelPtr == 2108 then
            if BoardPtr == 0 then
                GetAdjacentTiles(x)
            end
        end
        BoardPtr ← BoardPtr + 1
        PixelPtr ← PixelPtr + 1
    end
    if rf == 1 then
        FloodFill(x)
    end
end
```

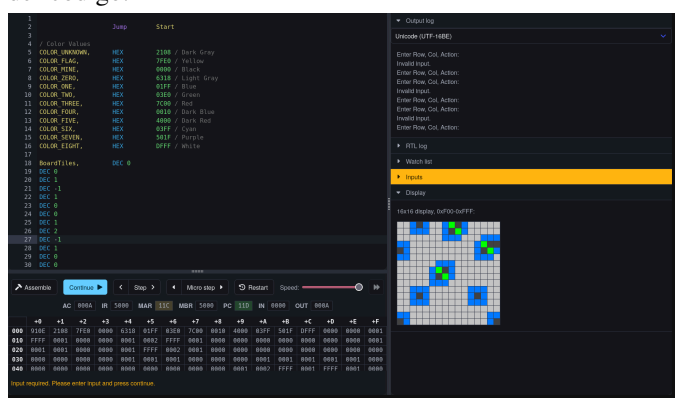
Verificación de victoria: CheckVictory compara el número de celdas reveladas (sr) con el total de celdas sin mina (SafeTiles). Si son iguales, el jugador gana.

Algoritmo 6. CheckVictory

```
▷ SafeTiles: total non-mine tiles, sr: revealed count
function CheckVictory(SafeTiles, sr)
    if SafeTiles > sr then
        return continue
    end
    if SafeTiles ≤ sr then
        return victory
    end
end
```

III. Resultados

A continuación se muestra una prueba del funcionamiento del codigo:



IV. Conclusiones

La implementación del juego Buscaminas en MARIE demuestra que es posible desarrollar aplicaciones interactivas relativamente complejas en una arquitectura educativa limitada. El uso de un script externo en Python para generar el tablero permite separar la generación de datos de la lógica del juego, siguiendo un patrón común en sistemas reales donde dispositivos externos preparan información para sistemas con recursos limitados. La visualización mediante colores en el display de 16×16 resulta efectiva para representar los estados del juego, aunque limita la capacidad de mostrar números directamente al usuario.

V. Anexos

Algoritmo 7. UpdatePixelPtr

```
▷ x: linear board index
▷ p: pixel RAM address (F00 + index)
function UpdatePixelPtr(x)
    p ← F00 + x
    return p
end
```

Algoritmo 8. GetBoardPtrCol

```
▷ p: pixel RAM address
▷ c: column index (0-15)
function GetBoardPtrCol(p)
    c ← p − D
    while c ≥ 16 do
        c ← c − 16
    end
    return c
end
```

Algoritmo 9. VerifyAdjacent

```
▷ p: pixel RAM address
▷ test: temporary pointer for checking
function VerifyAdjacent(p)
    if p < D then
        return null
    end
    if p ≥ 256 then
        return null
    end
    UpdatePixelPtr(test)
    if p == 2108 then
        RevealAdjacent(test)
        return true
    end
    return null
end
```

Algoritmo 10. RevealAdjacent

```
▷ p: pixel RAM address
▷ rf: reveal flag (1 = revealed), sr: safe revealed count
function RevealAdjacent(p)
    p ← 4 + p
    rf ← 1
    sr ← sr + 1
end
```

Algoritmo 11. InitBoard

```
▷ PixelPtr: current pixel RAM address
function InitBoard()
    while PixelPtr < 1000 do
        PixelPtr ← 2108
        PixelPtr ← PixelPtr + 1
    end
end
```

Algoritmo 12. GetAdjacentTiles

```
▷ x: linear index, c: column index
▷ BoardPtr: board RAM address
function GetAdjacentTiles()
    GetBoardPtrCol(x)
    if c == 0 then
        return skip
        AdjacentLeft(x)
    end
    if c == 15 then
        return skip
        AdjacentRight(x)
    end
    BoardPtr ← BoardPtr − 16
    VerifyAdjacent(x)
    BoardPtr ← BoardPtr + 32
    VerifyAdjacent(x)
end
```

Algoritmo 13. AdjacentLeft

```
▷ x: linear index, BoardPtr: board RAM address
function AdjacentLeft()
    BoardPtr ← BoardPtr − 1
    VerifyAdjacent(x)
    BoardPtr ← BoardPtr − 16
    VerifyAdjacent(x)
    BoardPtr ← BoardPtr + 32
    VerifyAdjacent(x)
    BoardPtr ← BoardPtr + 1
end
```

Algoritmo 14. AdjacentRight

```
▷ x: linear index, BoardPtr: board RAM address
function AdjacentRight()
    BoardPtr ← BoardPtr + 1
    VerifyAdjacent(x)
    BoardPtr ← BoardPtr − 32
    VerifyAdjacent(x)
    BoardPtr ← BoardPtr + 32
    VerifyAdjacent(x)
    BoardPtr ← BoardPtr − 1
end
```

Algoritmo 15. PrintScore

```
▷ sr: score to print, c: digit counter for BCD conversion
function PrintScore()
    c ← 0
    while sr ≥ 100 do
        c ← c + 1
        sr ← sr − 100
```

```
    end
    Output(c + 48)
    c ← 0
    while sr ≥ 10 do
        c ← c + 1
        sr ← sr − 10
    end
    Output(c + 48)
    Output(sr + 48)
end
```

References